

# Arai — Audit & Compliance Feature Inventory

How Arai turns instruction files you already have (CLAUDE.md, AGENTS.md, .cursorrules, and others) into enforceable policy, with a tamper-evident local evidence trail. Native PreToolUse hooks on Claude Code, Grok Build, and Codex; a git-diff gate for every other tool.

Current release: v1.1.2 (2026-09-16). This inventory supersedes the v0.2.18 document dated 2026-05-13.

Arai is not itself a certified product. This inventory documents controls and evidence so a procurement or InfoSec reviewer can evaluate Arai against their own framework. The certification is yours to pursue.

## 1. Evidence trail — every action is recorded, traceable, and queryable

Every guardrail firing is recorded on disk on the developer's own machine, traceable back to the line of policy that produced it. No log shipper required. Shipping to your own collector is an explicit administrative command, never on the hook hot path.

| Feature                               | What it does                                                                                                                                                                                                                                                                                                          | Why compliance teams care                                                                                                                                                |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Local JSONL audit log</b>          | One line per rule firing at <code>~/.taniwha/arai/audit/&lt;project&gt;/&lt;YYYYMMDD&gt;.jsonl</code> . Append-only, day-bucketed. Records timestamp, hook event, tool, session ID, prompt preview, decision (deny/inject/review), every matched rule. Owner-only on disk (0700/0600 Unix; icacis-pinned on Windows). | Tamper-evident record of every AI-assistant action where a guardrail evaluated, on the developer's own filesystem. Day-bucketed files map cleanly to retention policies. |
| <b>Derivation trace per firing</b>    | Each rule entry records the source file, line number, and parser layer that produced it (e.g. from CLAUDE.md:42, layer-1 imperative).                                                                                                                                                                                 | Auditors can answer "why did this rule fire?" without code spelunking. Maps every enforcement event back to a human-authored line of policy.                             |
| <b>Decision provenance</b>            | Each firing records the resolved decision (deny / inject / review) plus the severity that produced it.                                                                                                                                                                                                                | Distinguishes "advised but allowed" from "blocked" in evidence; supports policies like "all block-severity firings must be reviewed weekly."                             |
| <b>arai audit query CLI</b>           | Filter by <code>--since</code> , <code>--tool</code> , <code>--event</code> , <code>--outcome</code> , <code>--rule</code> (substring against subject/predicate/object). JSONL output for pipelines.                                                                                                                  | Investigations do not need a SIEM — grep and jq work directly on the local file.                                                                                         |
| <b>Hash-chained audit log</b>         | Every line carries <code>prev_hash + hash</code> (SHA-256 over canonical bytes); a per-day <code>.head.YYYYMMDD</code> sidecar anchors the chain tip. Concurrent writers serialize appends; incomplete tails are refused.                                                                                             | Tampering, reordering, or deletion of any line is detectable. Backs the tamper-evident claim with a mechanism, not append-only-by-convention.                            |
| <b>arai audit --verify</b>            | Walks the hash chain across every day-bucket and exits non-zero on any tamper / reorder / deletion. JSON output for CI / cron.                                                                                                                                                                                        | Evidence integrity becomes a gateable CI step or scheduled job. Auditors can independently confirm the trail has not been edited.                                        |
| <b>arai audit --ship</b>              | Opt-in POST of pending day-buckets plus chain-head sidecars to your HTTPS collector. Resume cursor, idempotent re-ship, optional bearer via <code>env-var</code> name. Never runs on the hook hot path.                                                                                                               | Centralizes the trail on your infrastructure (SIEM gateway, compliance collector) without making Arai a SaaS log shipper. Local-first remains the default.               |
| <b>Retention: arai audit --purge</b>  | <code>--older=N</code> drops day-buckets older than N days; <code>--project=&lt;slug&gt;</code> wipes a project. Today's bucket is always preserved; whole files are deleted so retained days keep a valid chain. <code>--dry-run</code> / <code>--json</code> for pre-purge review.                                  | Supports time-based retention policy and offboarding without hand-editing JSONL (which would break the hash chain).                                                      |
| <b>arai_recent_decisions MCP tool</b> | Read-only audit feed so the model can self-check what it was just denied for.                                                                                                                                                                                                                                         | Closes the model-side feedback loop; reduces deny-and-retry loops the audit log would otherwise show repeatedly.                                                         |

| Feature                           | What it does                                                                                | Why compliance teams care                                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>arai_check_action MCP tool</b> | Dry-run match of a hypothetical tool call against live rules. No execution, no audit write. | Lets MCP-only hosts (Cursor, Windsurf, Cline) probe policy before acting, without polluting the evidence trail with probes. |

## 2. Compliance verdicts — did the model honour the rule?

Pre/Post correlation generates evidence of enforcement effectiveness, not just enforcement attempts. Per-rule ratios surface which rules the model is honouring versus routing around.

| Feature                             | What it does                                                                                                                                                                          | Why compliance teams care                                                                                                     |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Pre/Post correlation</b>         | When PostToolUse fires, Arai correlates against recent PreToolUse firings in the same session and emits a Compliance audit event with an Obeyed / Ignored / Unclear verdict per rule. | Generates evidence of enforcement effectiveness, not just attempts. Answers “is the policy actually working?”                 |
| <b>First-definitive-wins dedupe</b> | One verdict per Pre firing — unrelated subsequent commands in the window cannot inflate the obeyed/ignored count.                                                                     | The headline number is statistically defensible. A maintainer saying “92% compliance on rule X” can stand behind that number. |
| <b>Per-rule compliance ratios</b>   | arai stats --by-rule rolls up fires / obeyed / ignored / unclear / ratio per rule, with a warning flag on low-ratio rules with enough volume.                                         | Single-glance answer to “which rules is the model routing around?” — the rules to rewrite or escalate.                        |
| <b>arai audit --outcome=ignored</b> | Shortcut filter for verdicts where the model ran the action despite the warning.                                                                                                      | Direct feed to a compliance review: every time a rule fired and the model ignored it this week.                               |

## 3. Controlled rollout — graduated enforcement, no all-or-nothing

Severity tiers map natural-language policy weight to enforcement strength. Individual rules can move into deny mode independently, with a project-wide kill switch as the rollback path.

| Feature                                   | What it does                                                                                                                                                             | Why compliance teams care                                                                                                                          |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Severity tiers</b>                     | never / forbids / must_not → Block (deny the tool call); always / requires / enforces → Warn; prefers / learned_from → Inform. Severity derived from the rule predicate. | Maps natural-language policy weight to enforcement strength deterministically. The writer’s grammatical choice (should vs must) sets the response. |
| <b>Per-rule severity override</b>         | arai severity <pattern> block warn inform pins individual rules in a dedicated DB column that survives arai scan re-classification.                                      | Incremental rollout: ship a rule set in advise mode, watch the ratios, escalate individual rules to deny once the model is honouring them.         |
| <b>Project-wide deny-mode kill switch</b> | ARAI_DENY_MODE=off forces advise-only for the whole rule set without changing any rule.                                                                                  | Rollback path that does not require editing policy files or restoring config from backup.                                                          |
| <b>arai disable / enable</b>              | Silence a rule by triple-id. The disable is keyed by content (subject/predicate/object) so it survives rescan. arai enable turns it back on.                             | Surgical override without editing the source policy file (which may be under SOX-style change control).                                            |
| <b>Rule expiry</b>                        | Trailing (expires YYYY-MM-DD) / (until YYYY-MM-DD) annotation; expired rules are filtered out at load.                                                                   | Time-boxed policies (incident-driven rules, migration windows) self-prune without manual cleanup.                                                  |

## 4. Policy authoring, distribution, and the git gate

Policy authoring tools that make instruction-file edits reviewable, testable, and distributable. The same matcher that runs in the hook also runs over a git diff, so a violating change is stopped whether an assistant or a human produced it.

| Feature                                                              | What it does                                                                                                                                                                                                                                                                                                                                          | Why compliance teams care                                                                                                                                                                   |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Shared policies via <code>arai:extends</code></b>                 | Inherit org-wide rules from a trusted upstream URL. Per-URL trust list, HTTPS only, 512 KB cap, 24-hour cache. SSRF-hardened (no loopback, RFC1918, link-local, cloud metadata; redirects disabled). Optional bearer token (env-var name, never written to disk) and optional ed25519 detached signature at <url>.sig.                                | Org-wide policy distribution without a hosted policy service. The trust list plus transport hardening covers the supply-chain checks an enterprise InfoSec team will ask.                   |
| <b>Extends <code>cache-at-rest</code> signature</b>                  | Cached upstream-policy files carry a SHA-256 sidecar. Mismatched or missing sidecars are a cache miss on both the fresh-read and stale-while-error paths.                                                                                                                                                                                             | An attacker with write access to the cache directory cannot swap a cached policy without the next read detecting it.                                                                        |
| <b><code>arai canonicalize</code> / <code>arai sync</code></b>       | <code>canonicalize</code> extracts recognised rules into a canonical <code>arai.toml</code> . <code>sync</code> writes that file into existing per-tool instruction files ( <code>CLAUDE.md</code> , <code>AGENTS.md</code> , <code>.cursorsrules</code> , ...) inside managed markers, without creating a tool file the team did not already choose. | One committed policy file instead of N drifting dialects. Extraction is lossy-but-honest; review <code>when.tool</code> / <code>when.path</code> before treating the TOML as authoritative. |
| <b><code>arai diff</code> / <code>lint</code> / <code>why</code></b> | <code>diff</code> previews rule-set delta before save. <code>lint</code> parses a file read-only. <code>why</code> replays a hypothetical tool call through the live match pipeline.                                                                                                                                                                  | Policy change review and QA before a <code>CLAUDE.md</code> edit lands. JSON output for pre-commit hooks.                                                                                   |
| <b><code>arai check-diff</code> / <code>pre-commit</code></b>        | Runs the live matcher on a git diff (stdin, <code>--cached</code> , or <code>--from-rev/--to-rev</code> ). Added files synthesise Write; modified/renamed files synthesise Edit. Block-severity matches exit 1. <code>arai init --pre-commit</code> installs a local git hook.                                                                        | Host hooks are per-tool. This gate is universal: Claude, Grok, Codex, Cursor, Copilot, or a human. Bypass is <code>git commit -no-verify</code> , like any local hook.                      |
| <b>Rule health checks</b>                                            | <code>arai status</code> flags duplicate rules (same SPO from multiple sources), opposing predicates, last firing (hooks registered but never invoked), and inert rules that will never fire.                                                                                                                                                         | Surfaces policy drift and dead rules at review time, not at firing time.                                                                                                                    |

## 5. Determinism and regression protection

Policy enforcement is testable. Regression tests, captured firings, and a pinned parser-coverage corpus prevent silent behavioural drift across releases.

| Feature                                        | What it does                                                                                                                                                                                              | Why compliance teams care                                                                                                          |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b><code>arai test scenario harness</code></b> | Replay synthetic hook payloads through the same <code>match_hook</code> pipeline the live hook handler uses. CI-friendly JSON output.                                                                     | Policy regression tests: every change to instruction files must keep specified scenarios passing. Pluggable into existing CI.      |
| <b><code>arai record</code></b>                | Build scenario files from recent audit-log entries — capture a real firing as a regression test.                                                                                                          | Turns observed enforcement into pinned tests with one command. The compliance team builds a regression corpus from real incidents. |
| <b>Parser-coverage corpus</b>                  | <code>tests/parser_coverage/</code> ships a synthetic instruction file exercising every pattern with positive and negative cases; <code>arai lint --json</code> is driven against it as a regression bar. | Guarantees the parser does not silently start under-extracting or over-extracting across releases.                                 |
| <b>Schema migration framework</b>              | PRAGMA <code>user_version</code> -tracked migrations in the store; migrations are append-only, idempotent, and crash-safe.                                                                                | Long-lived databases survive Arai upgrades without data loss. Critical for retention windows that span multiple releases.          |

## 6. Host coverage — where enforcement actually runs

Enforcement strength depends on the surface the assistant exposes. Arai does not invent a hook the host does not provide.

| Feature                                         | What it does                                                                                                                                                                                                                                                                                                                | Why compliance teams care                                                                                                                     |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Claude Code native hooks</b>                 | PreToolUse deny via permissionDecision. FileChanged / InstructionsLoaded auto-rescan; CwdChanged re-roots matching in a monorepo; PostToolBatch correlates parallel calls.                                                                                                                                                  | Strongest path: block plus advise, and the rule set stays current when instruction files change without a manual rescan.                      |
| <b>Grok Build native hooks</b>                  | Registers .grok/hooks/arai.json. Block is load-bearing (decision: deny + exit 2) when the host invokes hooks (verified on Grok Build 1.0.0 headless with --trust). Advise via additionalContext is best-effort. Project hooks are trust-gated.                                                                              | Same prohibitions actually deny on Grok, not only on Claude. Treat block as the guarantee; treat advise as optional context.                  |
| <b>Codex native hooks (v1.1.2)</b>              | Registers PreToolUse / PostToolUse / UserPromptSubmit in .codex/hooks.json. Shell calls use Codex's canonical Bash payload. apply_patch is checked per file (Add → Write, Update/Delete → Edit, Move checks the destination). Malformed patches fail closed. Enable via /hooks; writing the file does not grant host trust. | Codex writes files through apply_patch, not Write. Without per-file patch matching, creation rules would miss. Trust remains a host decision. |
| <b>MCP-only hosts (Cursor, Windsurf, Cline)</b> | Four tools: arai_add_guard, arai_list_guards, arai_check_action, arai_recent_decisions. Optional ARAI_MCP_AUTH_TOKEN at initialize. MCP does not inject or deny tool calls by itself.                                                                                                                                       | Honest coverage statement: advisory plus self-check on hosts without PreToolUse. Blocking still needs a native hook host or the git gate.     |
| <b>Hook binary path</b>                         | arai init registers the absolute path of the running binary (current_exe). Empty projects still get Claude, Grok, and Codex hook files so arai add rules have a live path.                                                                                                                                                  | A stale arai earlier on PATH cannot fail-open. Empty-repo onboarding does not silently skip enforcement.                                      |

## 7. Supply chain and operational hardening

Defensive controls covering binary distribution, upstream policy fetching, MCP input bounds, and the architectural separation of local audit data from anonymous telemetry.

| Feature                                 | What it does                                                                                                                                                           | Why compliance teams care                                                                                                                            |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SHA-256 checksum verification</b>    | install.sh and npm postinstall verify the downloaded binary against checksums.txt published with each GitHub release. Refuses to install on mismatch.                  | Default defense against a corrupted download. Does not by itself stop a substituted checksums.txt.                                                   |
| <b>Cosign keyless signatures</b>        | Every release binary is signed in CI with cosign keyless signing via GitHub OIDC (Fulcio). Verifiers pin --certificate-identity-regex to this repo's release workflow. | Answers "was this binary signed by this repo's CI?" No long-lived signing key to steal.                                                              |
| <b>SLSA L3 provenance</b>               | Releases include a <tag>.intoto.jsonl attestation from the SLSA GitHub generator. Verify with slsa-verifier pinned to github.com/taniwhaa/arai and the tag.            | Answers "how was this built — which commit, which workflow, which inputs?" Detects a tampered build pipeline even if the signing identity is intact. |
| <b>No network on the hook hot path</b>  | PreToolUse / PostToolUse consult only the local SQLite DB and audit JSONL. --ship, extends fetch, and telemetry flush are administrative or scan-time paths.           | Enforcement keeps working offline, in air-gapped environments, and during outages. No external service on the SLA for deny/allow.                    |
| <b>MCP input limits + optional auth</b> | arai_add_guard caps rule bodies at 1 KiB, reasons at 4 KiB, MCP-source rules at 1000 per project. Optional ARAI_MCP_AUTH_TOKEN (constant-time compare at initialize).  | Prevents a malfunctioning agent from filling the rule store. Closes the "agent on stdio is implicitly trusted" surface when the token is set.        |

| Feature                            | What it does                                                                                                                                                                                                | Why compliance teams care                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Anonymous opt-out telemetry</b> | Aggregate counters only — never file paths, rule text, code, or API keys. ARAI_TELEMETRY=off or DO_NOT_TRACK=1 opts out. Separate from the audit log. Optional self-hosted endpoint. Queue capped at 2 MiB. | The local audit log cannot leak via the telemetry channel — they share no code path. Privacy-policy-friendly distinction. |
| <b>Single-binary distribution</b>  | Lean default binary; optional full binary with ONNX local enrichment. No runtime dependencies. Hook commands use current_exe.                                                                               | Simplifies allowlisting / endpoint-management review. One artifact to track, one checksum to pin.                         |

## 8. Token economics — secondary signal

Arai's primary mission is correctness. The design also reduces token spend on long sessions and avoidable retry cycles. Numbers in arai stats are calibrated estimates with documented constants, not measurements.

| Feature                                         | What it does                                                                                                                                                | Why compliance teams care                                                                                                       |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Per-session repeat-injection suppression</b> | When a rule fires a second time in the same session, Arai emits a compact one-liner instead of the full payload.                                            | Reduces context-window pollution and token spend on long sessions; secondary signal.                                            |
| <b>Token-economics estimate in arai stats</b>   | Calibrated "tokens saved" estimate from suppressions, denied-and-honoured mistakes, and advised-and-honoured events. Labelled as estimate, not measurement. | Budget-conscious teams can quantify a contribution to LLM cost reduction; the constants are documented and auditable in source. |

## 9. SOC 2 Trust Service Criteria mapping

Arai is not itself a certified product. The mapping below documents how the controls in sections 1–8 align with the SOC 2 Trust Service Criteria so a reviewer can evaluate Arai against their own framework without redoing the analysis.

| TSC criterion                                 | What it covers                                                                                       | Arai control                                                                                                                                                                       |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CC6.1 — Logical access</b>                 | Restricts logical access to data and system resources to authorised users.                           | Audit directory locked to owner-only (~/.taniwha/arai/audit; 0700/0600 Unix; icacs /inheritance:r /grant:r on Windows, pinned on first write).                                     |
| <b>CC6.6 — External system / supply chain</b> | Protects information against vulnerabilities introduced by external sources or third-party software. | SHA-256 checksums on every install path; optional cosign keyless verification; optional SLSA L3 provenance. MCP input caps. Single-binary distribution.                            |
| <b>CC6.7 — Data in transit</b>                | Protects information in transit, including limits on what leaves the host.                           | No network on the hook hot path. arai:extends HTTPS-only, SSRF-hardened, optional bearer and ed25519. audit --ship is explicit opt-in to your collector, not a hidden egress path. |
| <b>CC7.2 — System monitoring</b>              | Detects security events through monitoring of system activity and configuration changes.             | Per-firing JSONL with derivation trace. SHA-256 prev_hash + hash; arai audit --verify walks every day-bucket and exits non-zero on tamper, reorder, or deletion.                   |
| <b>CC7.3 — Evaluation of security events</b>  | Evaluates events to determine whether the control environment is operating as designed.              | Pre/Post correlation emits Obeyed / Ignored / Unclear. arai stats --by-rule rolls these into per-rule ratios. Evidence of effectiveness, not just attempts.                        |

| TSC criterion                           | What it covers                                                                                                                             | Arai control                                                                                                                                                                                                                     |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CC8.1 — Change management               | Authorises, designs, develops, and tests changes prior to implementation.                                                                  | arai diff previews rule-set delta. arai test replays scenarios through match_hook. arai record captures real firings. check-diff gates violating diffs at commit. Parser-coverage corpus locks parser behaviour across releases. |
| CC9.2 — Vendor / third-party management | Manages risks associated with vendors and business partners (the agent that drives Arai's MCP server, and shared upstream policy sources). | Optional MCP shared-secret. arai:extends per-URL trust list, optional authenticated fetch, SHA-256 cache sidecars, optional ed25519 URL signatures.                                                                              |

## Suggested enterprise positioning

A short version of the matrix above for a sales or procurement-review summary deck:

1. Local-first audit log — every guardrail firing recorded on disk at `~/.taniwha/arai/audit`, traceable to the source instruction-file line, queryable without external services. Optional `--ship` to your collector.
2. Compliance verdicts — every `PostToolUse` correlated against `PreToolUse` to produce per-rule obeyed/ignored ratios. Evidence of enforcement effectiveness, not just attempts.
3. Graduated enforcement — severity tiers (Block / Warn / Inform) derived from policy language. Per-rule overrides, disable/enable, and a project-wide kill switch.
4. Native hooks on Claude Code, Grok Build, and Codex (v1.1.2), plus a git check-diff gate for every other tool. MCP-only hosts get lookup, add, dry-run check, and recent decisions — not automatic deny.
5. Org-wide policy distribution — `arai:extends` with an explicit per-URL trust list, SSRF-hardened transport, optional bearer and signatures.
6. Regression-tested policy — `arai test`, `arai record`, `check-diff`, and the parser-coverage corpus turn rule changes into CI assertions.
7. No data egress on the hot path — opt-out telemetry architecturally separate from the audit log. Supply-chain hardened: SHA-256, optional cosign, optional SLSA L3.